CAPSTONE PROJECT

AUTOSAR HLD Intelligence & Traceability Assistant

Technical Report

SASTRA Deemed University

Student Name: Guru S

Register Number: 127003080

Programme / Department: B.Tech CSE / School of Computing

Submission Date: 13-10-2026

Executive summary

The AUTOSAR HLD Intelligence & Traceability Assistant is an AI-assisted engineering application intended to reduce the effort required to inspect high-level design (HLD) documents. It combines document extraction, architecture-aware chunking and metadata, vector retrieval, retrieval-augmented generation (RAG), source citations, deterministic validation, revision comparison, and human review. The interface uses Streamlit and the service layer uses FastAPI. The documented local model configuration uses Ollama with Qwen2.5:3B for generation and BAAI/bge-small-en-v1.5 embeddings with FAISS for retrieval. SQLite is used for local workflow/governance data and Docker is used to package the services.

The current demonstration uses synthetic/versioned HLD fixtures named autosar_hld_v1.0 and autosar_hld_v2.0. These fixtures allow repeatable tests such as checking a signal data type across revisions. They must not be presented as genuine OEM engineering specifications. The system is an engineering aid: engineers remain responsible for validating evidence and approving decisions.

Contents

1. Problem statement and intended users | 5. Evaluation approach and results
2. Input data and knowledge base | 6. Responsible AI, privacy and security
3. Solution architecture and workflow | 7. Innovation highlights
4. Implementation and configuration | 8. Limitations and future work
Appendices A-B: Reproducibility checklist and folder placement

1. Problem statement and intended users

1.1 Problem statement

High-level design documents can contain descriptions of software components, interfaces, ports, signals, dependencies, and functional flows across many pages. Engineers may need to search manually for a single detail, establish where a statement came from, or compare revisions to understand potential impact. Manual review can be slow and findings may be difficult to trace back to page- and section-level evidence.

The project objective is to provide a searchable, evidence-linked engineering assistant that helps organize architectural knowledge and supports review. It is designed to reduce manual lookup effort, improve traceability, surface possible inconsistencies, and make revision analysis easier. It does not automatically approve or modify an HLD.

1.2 Intended users

- System and AUTOSAR architects reviewing component structure and interfaces.
- Software developers and integration engineers tracing ports, signals, and dependencies.
- Test engineers using identified changes or gaps to inform test planning.
- Engineering reviewers and managers who need evidence-backed summaries and structured findings.

1.3 Scope

In scope	Not claimed / outside current authority
PDF ingestion, text and table extraction, scanned-page OCR fallback where supported.	Automatic approval of architecture decisions.
Architecture-aware chunks and source metadata for document/version/page/section traceability.	Replacement for expert design review or formal organizational governance.
Natural-language Q&A with retrieved evidence and citations.	Proof of compliance with AUTOSAR, ISO 26262, or any safety standard.
Architecture validation, inconsistency checks, revision comparison, and review workflow.	Unreviewed mutation of source HLD documents.

2. Input data and knowledge base

2.1 Inputs used in the current demonstration

The known demonstration fixtures are versioned documents labeled autosar_hld_v1.0 and autosar_hld_v2.0. The test data is synthetic. The version distinction supports deterministic retrieval and cross-version questions. For example, in the current synthetic fixtures the VehicleSpeed signal is represented as float32 in version 1.0 and float64 in version 2.0. This is a test fixture fact, not a statement about any real vehicle or OEM design.

2.2 Knowledge-base preparation

1. Accept an approved input PDF and retain its document identity and version.
2. Extract text, headings, page-level content, and tables where possible.
3. Use OCR fallback for scanned pages when normal text extraction is insufficient and OCR is available/configured.
4. Split content into structure-aware chunks while retaining source document, version, page, section, and chunk identifiers.
5. Generate text embeddings with BAAI/bge-small-en-v1.5 and index vectors in FAISS.
6. Use retrieved chunks as evidence for RAG generation and citation validation.

2.3 Source traceability

Retrieval and answer records include chunk identifiers and citation metadata such as document, version, page, and section. Keeping this provenance attached to each chunk makes it possible for a reviewer to open the supporting source and check whether a claim is properly attributed. A citation being syntactically valid does not by itself guarantee that the claim is correct; claim-to-evidence checking is therefore also required.

3. Solution architecture and workflow

The architecture separates the user interface, API/orchestration layer, document processing, embedding/retrieval, language-model generation, deterministic analysis, and evidence/review outputs. The diagram below is a high-level representation; verify its labels against the exact final repository before submission.

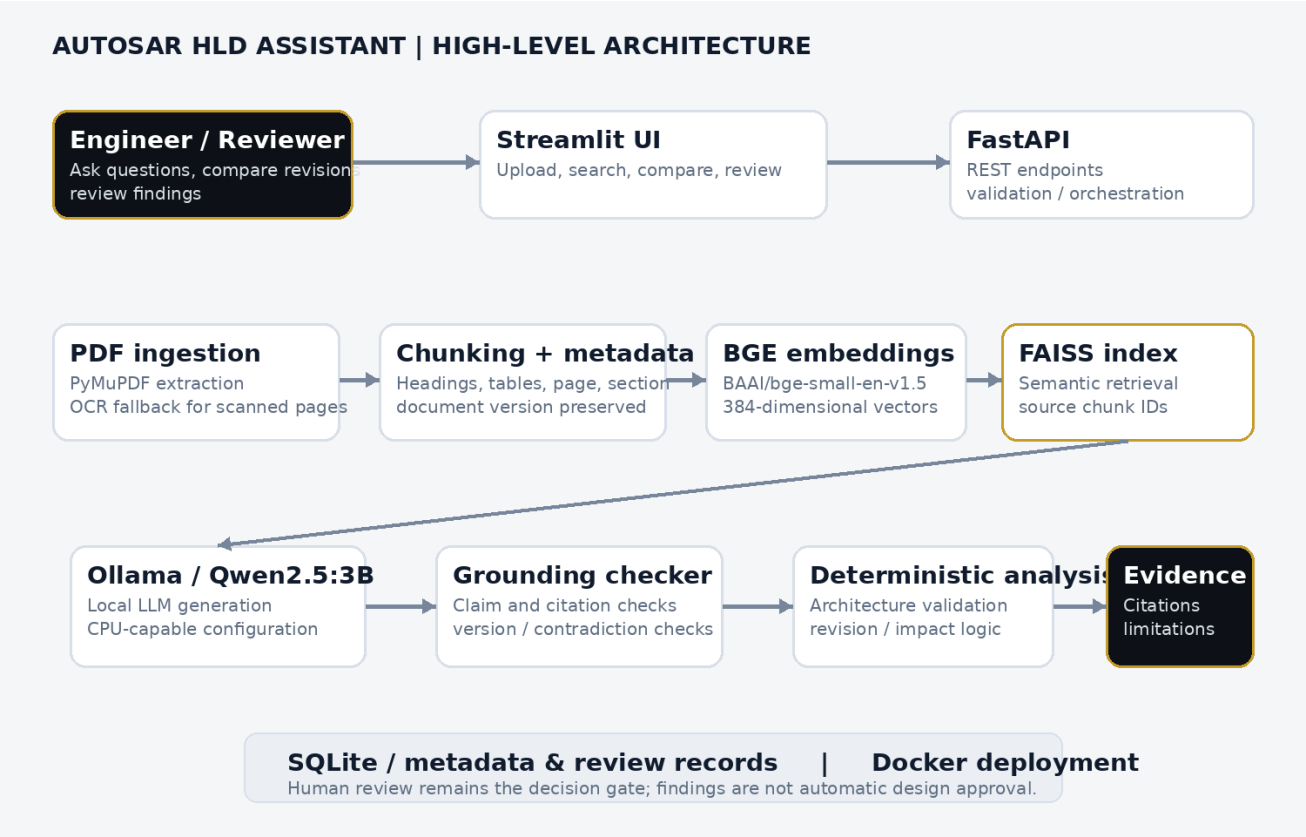


Figure 1. High-level architecture of the AUTOSAR HLD Assistant.

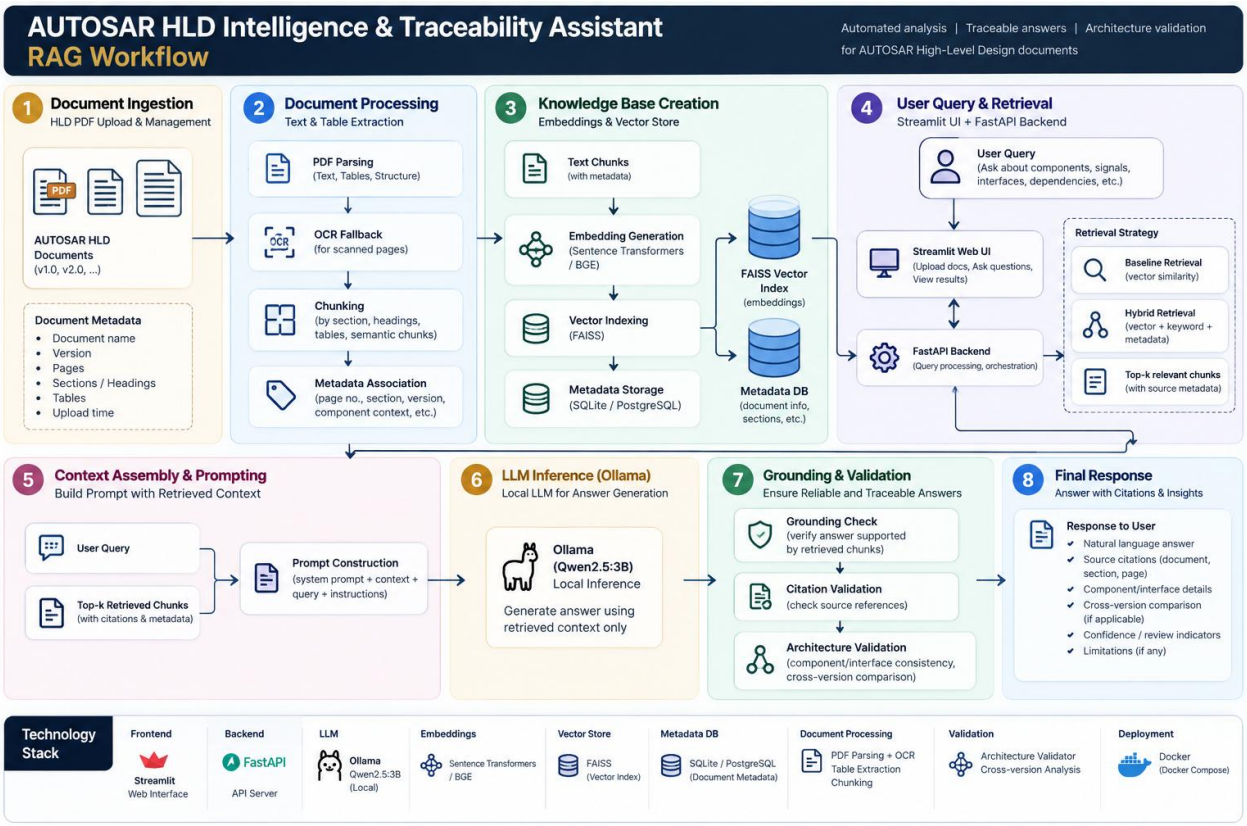


Figure 2. Detailed RAG workflow from PDF ingestion through retrieval, local inference, grounding checks, citations, and human review.

3.1 Runtime Q&A workflow

1. The engineer submits a question through the Streamlit interface.
2. The API routes the request into retrieval and analysis logic.
3. The query is embedded and compared with indexed chunks; the retriever returns the configured top_k evidence set.
4. The prompt supplies retrieved evidence to the configured local LLM through Ollama.
5. The response is checked for evidence support, version alignment, citation validity, and contradictions where applicable.
6. The UI presents the answer, citations, confidence/grounding indicators, limitations, and whether review is required.
7. A human reviewer checks the cited document sections before relying on the answer for engineering decisions.

3.2 Revision and architecture analysis

Version comparison and architecture validation use structured document metadata and deterministic Python logic for the checks implemented in the project. These results are intended to identify candidate changes or inconsistencies for review, not to establish semantic correctness without supporting evidence.

4. Implementation and configuration

Layer	Known project choice	Role
User interface	Streamlit	Document selection/upload, question entry, result and review views.
API	FastAPI / Uvicorn	HTTP endpoints and orchestration between UI, retrieval, analysis, and persistence.

Layer	Known project choice	Role
PDF extraction	PyMuPDF (`fitz` API in current logs)	Extract PDF text and page information; OCR fallback for scanned documents where configured.
Embeddings	BAAI/bge-small-en-v1.5	Encode query and document chunks into 384-dimensional vectors.
Vector index	FAISS	Find candidate chunks by vector similarity; architecture-aware hybrid logic may add additional signals.
LLM inference	Ollama / Qwen2.5:3B	Generate responses grounded in retrieved context.
Structured storage	SQLite (local workflow/governance)	Store local structured metadata and review/workflow records.
Deployment	Docker Compose	Run the API/UI services; container-to-host Ollama connectivity uses `host.docker.internal` in the described Windows setup.

4.1 Configuration principles

- Keep model name, inference timeout, and retrieval parameters in configuration rather than hard-coding them throughout the application.
- Use an environment template with placeholder values for shareable configuration. Do not commit or submit the real `.env` file if it contains secrets.
- Keep the embedding model consistent between index construction and query-time retrieval. Rebuild the index if embedding configuration or vector dimensions change.
- Use the same top_k value when comparing Baseline and Hybrid retrieval so the evaluation has a consistent evidence budget.
- Record the code commit, configuration, dataset versions, and commands for every evaluation run.

4.2 Key logical modules

Module / area	Responsibility (verify final paths)
`rag/hybrid_retriever.py`	Hybrid retrieval and scoring logic.
`rag/llm_provider.py`	LLM provider integration and response generation.
`rag/prompts.py`	System/user prompt instructions for grounded answers.
`rag/grounding_checker.py`	Claim grounding, source/version alignment, and contradiction checks.
`analysis/architecture_validator.py`	Deterministic architecture validation rules.
Chunking, embedding, vector-store, and metadata modules	Document chunk preparation, vector indexing, and retrieval metadata.

5. Evaluation approach and results

5.1 Evaluation dimensions

- Answer correctness against a known expected answer.
- Retrieval relevance: whether the returned chunks contain the evidence needed for the question.
- Citation validity: whether each cited chunk exists in the retrieved set and metadata matches its source.
- Grounding: whether individual answer claims are supported by cited evidence.

- Cross-version fidelity: whether claims are attributed to the correct document version.
- Latency and stability across repeated runs.
- Regression tests for grounding, architecture validation, retrieval, API configuration, and deployment.

5.2 Recorded results available at drafting time

Test / observation	Recorded result	Interpretation
Focused automated test suite	63 passed in 28.65 seconds across the reported focused test command.	Positive evidence for the selected regression tests only; rerun against the final submitted commit.
VehicleSpeed cross-version RAG, two newer saved reports	Both reports answer v1.0 = float32 and v2.0 = float64; each reports grounding_score 1.0, high confidence, review_required false, 5 retrieved chunks and valid citations.	Successful individual runs. These do not establish a stable success rate.
Earlier VehicleSpeed reports	Earlier saved runs included incorrect v2.0 claims, contradictory evidence, and/or a citation metadata mismatch.	Known intermittent failure mode; must be discussed rather than hidden.
Preliminary Baseline vs Hybrid retrieval-only comparison	HitRate@5: Baseline 1.000, Hybrid 1.000. Recall@5: Baseline 0.5833, Hybrid 0.8333 (+0.2500 absolute). MRR: Baseline 0.7500, Hybrid 1.0000 (+0.2500 absolute).	Reported preliminary retrieval-only run: Hybrid improves recall and reciprocal rank while hit rate is unchanged. These metrics do not measure generated-answer correctness; verify raw outputs and rerun against the final commit.
Latency	One described local Qwen2.5:3B RAG run took about 73 seconds for a roughly 1,750-token prompt and a 287-token response.	Single observation on CPU-only environment, not a benchmark or general SLA.

Important interpretation. The Baseline-versus-Hybrid figures are preliminary retrieval-only results reported during project work; verify them against saved raw metrics and rerun them on the exact submitted commit. They are not answer-accuracy or RAGAS scores. Earlier local Qwen2.5:3B runs showed intermittent cross-version errors, even though some later saved reports were correct. A full live RAGAS evaluation with the Ollama judge remains outstanding unless its raw outputs are generated and added. Do not treat mock-judge or smoke-test results as quality scores. Report both successes and failures transparently.

5.3 Recommended repeatable comparison protocol

1. Freeze a test set of questions with expected answers and expected source versions.
2. Run Baseline and Hybrid with the same top_k, dataset/index, embedding model, LLM, prompt, and generation settings.
3. Repeat each question several times to measure generation variability; retain raw JSON reports.
4. For each run, score correctness, relevant-source retrieval, citation validity, grounding/contradiction, latency, and review flags.
5. Report aggregate counts and percentages with the run count in the denominator. Include failures and unresolved edge cases.
6. Re-run the same tests after changing prompts, retrieval scoring, or grounding checks.

6. Responsible AI, privacy and security

6.1 Grounding and traceability

The RAG design conditions generation on retrieved HLD excerpts, preserves chunk metadata, and displays citations. Claim-level checks seek to detect unsupported statements, contradictory values, and document-version mismatches. Scores and labels such as “grounded” are software signals, not a mathematical guarantee of truth. Human inspection of the cited source remains necessary.

6.2 Privacy and local processing

The described configuration uses a local Ollama model and local vector retrieval, which can reduce the need to send HLD content to an external model API. Local execution does not by itself make the deployment secure: uploaded documents, logs, Docker ports, backups, and stored credentials still require appropriate protection. Only documents approved for the intended environment should be ingested.

6.3 Secret management and access

- Keep API keys and credentials out of source control, PDFs, screenshots, logs, and evaluation exports.
- Submit `.env.example` with placeholders only; exclude `.env` from the ZIP/repository when it contains secrets.
- If a credential is exposed, revoke or rotate it and update the local configuration.
- Restrict document access and ensure citations point only to documents the current user is authorized to view, if access control is part of the deployment.
- Retain request IDs and useful error logs without logging secrets or unnecessarily copying sensitive HLD content.

6.4 Human oversight and scope of claims

The system supports human review and should not independently approve an architecture change or make a safety decision. It has not been represented here as certified, safety-rated, or compliant with AUTOSAR or ISO 26262. Any finding needs engineering interpretation and confirmation against authoritative source documents and organizational processes.

7. Innovation highlights

- Architecture-aware RAG for engineering documents rather than generic PDF chat alone.
- Citation and metadata preservation to support page/section-level traceability.
- Cross-version rules to reduce the risk of assigning a fact from one HLD revision to another.
- Claim-level grounding and contradiction checking to catch plausible but incorrect generated answers.
- Deterministic validation and revision-analysis logic paired with an LLM, keeping checks that can be explicit in code separate from open-ended generation.
- Human review flags and governance workflow intended to keep engineers accountable for final decisions.

8. Limitations and future work

- Cross-version answers have shown intermittent errors in earlier runs; reliability needs a larger, repeatable evaluation set.
- The quality of extraction and table parsing depends on the layout and quality of the source PDF; scanned or complex tables may need OCR or manual verification.
- Local CPU inference may have high latency for long prompts and generated answers.
- The demonstration fixtures are synthetic and do not prove performance on confidential, real-world OEM HLDs.
- The preliminary retrieval-only comparison is not a complete end-to-end answer-quality evaluation; a controlled live RAGAS run remains to be verified.
- Future work: broaden evaluation data, add deterministic schema/row-aware checks, improve retrieval ranking, benchmark latency, and validate access-control behavior with representative authorized documents.

Conclusion

The AUTOSAR HLD Intelligence & Traceability Assistant brings together document ingestion, local retrieval-augmented generation, source citations, architecture checks, revision analysis, and human review in a deployable prototype. The strongest next step before submission is to rerun a fixed evaluation suite on the exact final build, preserve the resulting reports, and present both successes and known failure cases. The system should be described as an engineering assistant that supports traceable review, not as a replacement for expert approval.